Tecnológico Superior de Jalisco – Unidad La Huerta

Carrera: Ingeniería en Sistemas Computacionales

Materia: Ingeniería de Software

Actividad 2: Completar el esquema de DB

Docente: Omar Gerardo Pérez Morales

Alumnos:

- Williams Díaz
- Ali Grijalva
- Víctor Sainz
- Ernesto Reynaga
- Brandon Ramos
- Raúl Beltrán

Fecha: 28/02/2026

Modelo de Datos y Normalización

El modelo de datos fue diseñado bajo los principios de normalización para garantizar integridad referencial, reducción de redundancia y consistencia estructural.

El sistema cumple con la **Primera, Segunda, Tercera y Cuarta Forma Normal**.

10.4.1 Primera Forma Normal (1FN)

Se garantiza que:

- Cada tabla posee una clave primaria definida.
- No existen atributos multivaluados.
- Cada campo almacena un único valor (atomicidad).

Ejemplo aplicado:

La relación entre préstamos y materiales no se almacena en un solo campo. En su lugar, se utiliza la tabla “Detalle_Prestamo”, evitando almacenar múltiples materiales en una sola columna.

10.4.2 Segunda Forma Normal (2FN)

Se eliminan dependencias parciales.

Todos los atributos no clave dependen completamente de su clave primaria.

Ejemplo:

En la tabla **Prestamos**, los atributos:

- fecha_prestamo
- fecha_devolucion
- estado

dependen únicamente de id_prestamo.

Asimismo, en “Detalle_Prestamo”, la cantidad depende directamente del identificador del detalle y de la relación específica entre préstamo y material.

10.4.3 Tercera Forma Normal (3FN)

Se eliminan dependencias transitivas.

Ejemplo:

La información de roles se encuentra separada en la tabla **Roles**, evitando que el nombre del rol dependa indirectamente del usuario.

Usuarios:

- id_usuario (PK)
- nombre
- correo
- id_rol (FK)

Roles:

- id_rol (PK)
- nombre_rol

Esto evita redundancia y facilita modificaciones futuras.

Se eliminan dependencias multivaluadas independientes.

Ejemplo:

Teléfonos

- id_telefono
- id_usuario
- telefono

Correos

- id_correo
- id_usuario
- correo

El modelo contempla que, en caso de múltiples teléfonos o correos por usuario, estos se almacenen en tablas independientes, evitando almacenar múltiples valores en una sola columna.

10.5 Modelo Relacional Final

El modelo relacional final integra todas las entidades necesarias para cubrir los requerimientos funcionales del sistema.

10.5.1 Usuarios

- id_usuario (PK)
- nombre
- correo
- contraseña
- id_rol (FK)

10.5.2 Roles

- id_rol (PK)
- nombre_rol

10.5.3 Materiales

- id_material (PK)
- nombre
- descripcion
- cantidad_disponible
- id_categoria (FK)

10.5.4 Categorias

- id_categoria (PK)
- nombre_categoria

10.5.5 Prestamos

- id_prestamo (PK)
- id_usuario (FK)
- fecha_prestamo
- fecha_devolucion
- estado

10.5.6 Detalle_Prestamo

- id_detalle (PK)
- id_prestamo (FK)
- id_material (FK)
- cantidad

10.5.7 Reservas

- id_reserva (PK)
- id_usuario (FK)
- fecha_reserva
- estado

10.5.8 Bitácora

- id_bitacora (PK)
- id_usuario (FK)
- accion
- fecha

10.6 Relaciones Principales

Las relaciones del modelo son:

- Roles 1:N Usuarios
- Usuarios 1:N Prestamos
- Prestamos 1:N Detalle_Prestamo
- Materiales 1:N Detalle_Prestamo
- Usuarios 1:N Reservas
- Usuarios 1:N Bitacora
- Categorias 1:N Materiales

DICCIONARIO DE DATOS

Sistema de Control de Inventario de Laboratorios

Tabla: users

Descripción

La tabla users almacena la información de las personas que interactúan con el sistema. Representa a los distintos actores que pueden realizar operaciones dentro de la plataforma, como solicitudes de préstamo, reservas de laboratorio y administración de inventario.Esta tabla es fundamental para el control de acceso y la trazabilidad, ya que cada acción realizada en el sistema queda vinculada a un usuario específico mediante claves foráneas en otras tablas como requests, reservations y logs.

Campo	Tipo de Dato	Restricciones	Descripción
id	integer	PK, NOT NULL	Identificador único del usuario dentro del sistema
username	varchar	NOT NULL, UNIQUE	Nombre de usuario utilizado para autenticación
password_hash	varchar	NOT NULL	Contraseña en formato cifrado
email	varchar	UNIQUE	Correo electrónico institucional o personal
role	user_role (ENUM)	DEFAULT 'alumno'	Define el nivel de acceso (alumno, docente, admin)
created_at	timestamp	DEFAULT CURRENT_TIMESTAMP	Fecha de registro en el sistema

Tabla: categories

Descripción

La tabla categories permite clasificar los recursos del laboratorio en grupos organizados, facilitando la gestión y consulta del inventario.

Se utiliza tanto para activos como para consumibles, diferenciándolos mediante el campo type. Esto evita duplicar estructuras y mejora la normalización del sistema.

Su propósito principal es mantener el inventario estructurado y permitir filtros, reportes y estadísticas organizadas por categoría.

Campo	Tipo de Dato	Restricciones	Descripción
id	integer	PK, NOT NULL	Identificador único de la categoría
name	varchar	NOT NULL	Nombre representativo de la categoría
description	text	—	Información detallada sobre la categoría
type	varchar	CHECK ('asset','consumable')	Indica si pertenece a activos o consumibles

Tabla: assets

Descripción

La tabla assets almacena los bienes físicos del laboratorio que pueden ser prestados o asignados temporalmente a los usuarios.
Incluye equipos como computadoras, proyectores, impresoras, cámaras o cualquier recurso reutilizable. El campo status permite controlar el estado operativo del activo (disponible, prestado o en mantenimiento), mientras que serial_number facilita la identificación única del equipo.
Esta tabla se relaciona directamente con requests, permitiendo registrar qué activo fue solicitado y su estado dentro del flujo de préstamo.

Campo	Tipo de Dato	Restricciones	Descripción
id	integer	PK	Identificador único del activo
name	varchar	NOT NULL	Nombre del equipo o recurso
description	text	—	Especificaciones técnicas o detalles adicionales
category_id	integer	FK → categories(id)	Clasificación del activo
serial_number	varchar	UNIQUE	Código único del fabricante
location	varchar	—	Ubicación física dentro del laboratorio
status	varchar	CHECK	Estado operativo actual
quantity	integer	CHECK >=0	Cantidad disponible en inventario
created_at	timestamp	DEFAULT CURRENT_TIMESTAMP	Fecha de alta
updated_at	timestamp	DEFAULT CURRENT_TIMESTAMP	Última modificación

Tabla: consumables

Descripción

La tabla consumables registra los materiales que se agotan con el uso, como hojas, tinta, cables, baterías, etc.

A diferencia de los activos, estos no se devuelven, sino que su inventario disminuye conforme se autorizan solicitudes.

El campo min_quantity permite establecer un nivel mínimo de stock, facilitando la generación de alertas para reposición.

Campo	Tipo de Dato	Restricciones	Descripción
id	integer	PK	Identificador único
name	varchar	NOT NULL	Nombre del material
description	text	—	Características o especificaciones
category_id	integer	FK → categories(id)	Clasificación
quantity	integer	CHECK >=0	Cantidad actual disponible
min_quantity	integer	CHECK >=0	Nivel mínimo de alerta
unit	varchar	DEFAULT 'units'	Unidad de medida
created_at	timestamp	DEFAULT CURRENT_TIMESTAMP	Fecha de registro
updated_at	timestamp	DEFAULT CURRENT_TIMESTAMP	Fecha de actualización

Tabla: requests

Descripción

La tabla requests representa el núcleo operativo del sistema. Registra todas las solicitudes realizadas por los usuarios para obtener activos o consumibles.

Incluye el flujo completo del proceso: solicitud, aprobación o rechazo, y devolución en caso de activos.

Permite llevar control histórico de préstamos y consumo de recursos, siendo clave para reportes administrativos y auditoría.

Campo	Tipo de Dato	Restricciones	Descripción
id	integer	PK	Identificador de la solicitud
user_id	integer	FK → users(id)	Usuario solicitante
asset_id	integer	FK → assets(id)	Activo solicitado (si aplica)
consumable_id	integer	FK → consumables(id)	Consumible solicitado (si aplica)
quantity_requested	integer	CHECK >0	Cantidad solicitada
status	varchar	CHECK	Estado del proceso
request_date	timestamp	DEFAULT CURRENT_TIMESTAMP	Fecha de creación
approval_date	timestamp	—	Fecha de autorización
return_date	timestamp	—	Fecha de devolución (activos)
notes	text	—	Comentarios administrativos

Tabla: reservations

Descripción

La tabla reservations gestiona la programación de uso de laboratorios por parte de los usuarios.

Permite registrar horarios específicos de uso, evitando conflictos de disponibilidad.

Es esencial para organizar actividades académicas, prácticas o eventos especiales dentro de las instalaciones.

Campo	Tipo de Dato	Restricciones	Descripción
id	integer	PK	Identificador único
user_id	integer	FK → users(id)	Usuario que reserva
lab_name	varchar	NOT NULL	Laboratorio reservado
start_time	timestamp	NOT NULL	Inicio de la reserva
end_time	timestamp	NOT NULL	Finalización
status	varchar	CHECK	Estado de la reserva
purpose	text	—	Motivo o actividad
created_at	timestamp	DEFAULT CURRENT_TIMESTAMP	Fecha de registro

7 Tabla: logs

Descripción

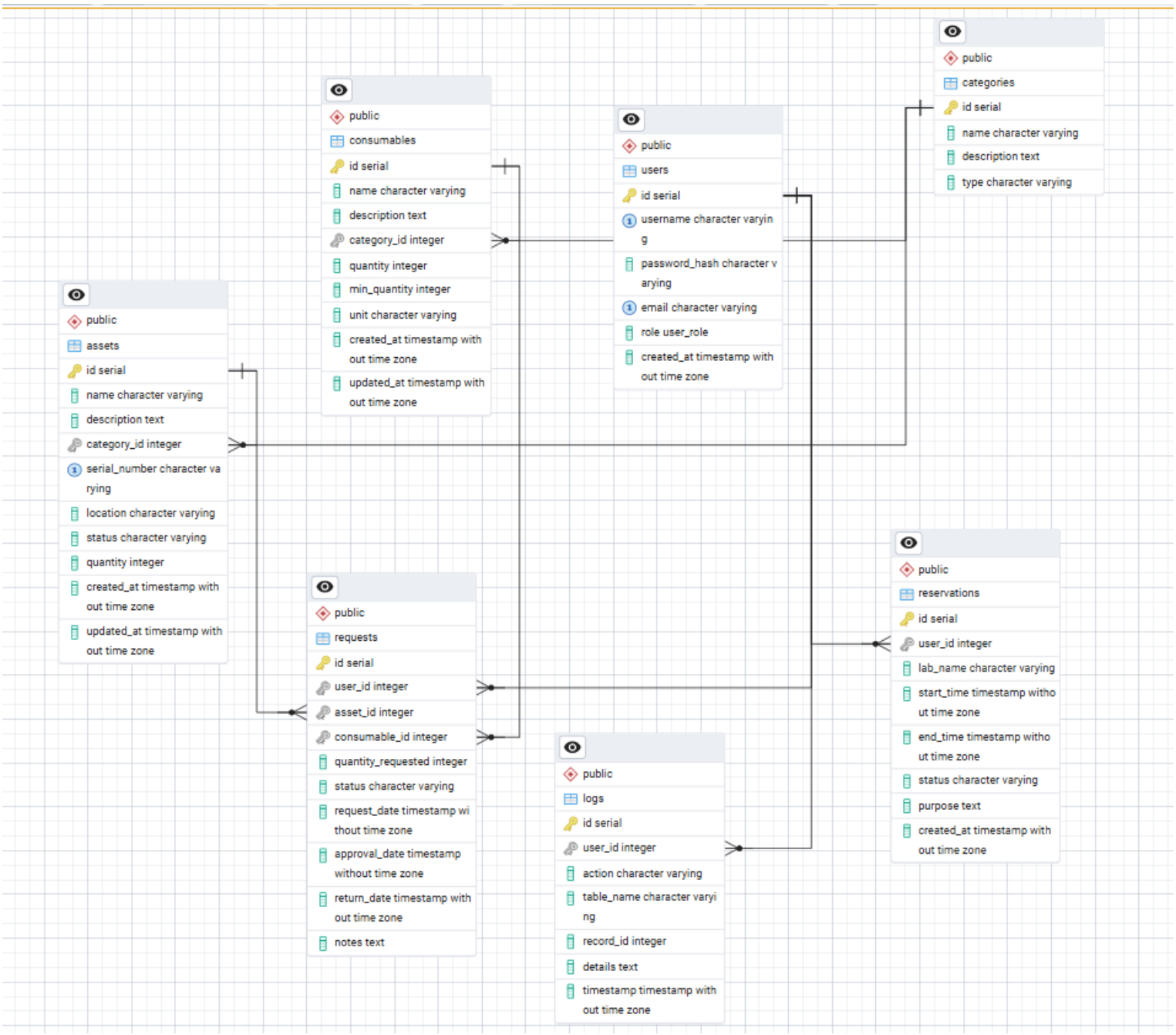
La tabla logs permite implementar un sistema de auditoría interna.

Registra todas las acciones relevantes realizadas por los usuarios, como creación, actualización o eliminación de registros.

Es fundamental para garantizar transparencia, seguridad y trazabilidad en el sistema.

Campo	Tipo de Dato	Restricciones	Descripción
id	integer	PK	Identificador del evento
user_id	integer	FK → users(id)	Usuario que ejecutó la acción
action	varchar	NOT NULL	Tipo de operación (INSERT, UPDATE, DELETE, etc.)
table_name	varchar	—	Tabla afectada
record_id	integer	—	ID del registro modificado
details	text	—	Información detallada del cambio
timestamp	timestamp	DEFAULT CURRENT_TIMESTAMP	Fecha y hora del evento

ESQUEMA DE BASE DE DATOS



INFORME SOBRE EL ESPACIO UTILIZADO EN LA BASE DE DATOS

Sistema de Gestión de Inventario

1 Introducción

El presente informe describe el espacio de almacenamiento utilizado por los distintos elementos que componen la base de datos del Sistema de Gestión de Inventario, incluyendo tablas, campos, índices y relaciones.

El análisis se realizó utilizando herramientas internas de PostgreSQL a través de pgAdmin, permitiendo identificar el tamaño físico ocupado por cada objeto dentro del motor de base de datos.

2 Espacio total de la Base de Datos

Para consultar el tamaño total de la base de datos se utiliza:

```
SELECT pg_size_pretty(pg_database_size('SUPER'));
```

Esta función devuelve el tamaño total incluyendo:

- Tablas
- Índices
- Secuencias
- TOAST
- Metadatos

RESULTADO QUE SE MUESTRA:

	pg_size_pretty text
1	8470 kB

Espacio utilizado por cada tabla

Consulta utilizada:

```sql
SELECT
    relname AS tabla,
    pg_size_pretty(pg_total_relation_size(relid)) AS tamaño_total
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;
```

Esta consulta muestra:

- Tamaño de datos
- Índices
- Información auxiliar

	tabla name	tamaño_total text
1	users	32 kB
2	assets	24 kB
3	consumab...	16 kB
4	categories	16 kB
5	requests	16 kB
6	reservatio...	16 kB
7	logs	16 kB

Espacio ocupado solo por datos (sin índices)

Consulta utilizada:

```sql
SELECT
    relname AS tabla,
    pg_size_pretty(pg_relation_size(relid)) AS tamaño_datos
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_relation_size(relid) DESC;
```

RESULTADO QUE SE MUESTRA:

	tabla name	tamaño_datos text
1	categories	0 bytes
2	assets	0 bytes
3	consumab...	0 bytes
4	users	0 bytes
5	requests	0 bytes
6	reservatio...	0 bytes
7	logs	0 bytes

Espacio ocupado por índices

Consulta utilizada:

```
SELECT
    relname AS tabla,
    pg_size_pretty(pg_indexes_size(relid)) AS tamaño_indices
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_indexes_size(relid) DESC;
```

Resultado que muestra:

	tabla name	tamaño_indices text
1	users	24 kB
2	assets	16 kB
3	consumab...	8192 bytes
4	categories	8192 bytes
5	requests	8192 bytes
6	reservatio...	8192 bytes
7	logs	8192 bytes

Análisis por tipo de elemento

Tablas principales que generan mayor almacenamiento:

- requests (historial acumulativo)
- logs (registro continuo de auditoría)
- assets
- consumables

Estas tablas tienden a crecer con el tiempo debido a:

- Registro histórico de movimientos
- Auditoría del sistema
- Control de préstamos

Índices

Los índices ocupan espacio adicional, pero mejoran significativamente:

- Velocidad de búsqueda
- Consultas por claves foráneas
- Filtros por estado o usuario

Ejemplo de índices generados automáticamente:

- Claves primarias
- Claves foráneas
- Campos UNIQUE

Relaciones

Las relaciones no ocupan espacio como entidad física independiente, pero sí:

- Generan índices implícitos
- Mantienen integridad referencial
- Añaden validaciones internas

Consideraciones Técnicas

El espacio utilizado depende de:

- Número de registros almacenados
- Tipo de datos (VARCHAR, TEXT, INTEGER, TIMESTAMP)
- Longitud promedio de cadenas
- Cantidad de índices creados
- Crecimiento histórico de logs y solicitudes

En etapas iniciales, el tamaño es reducido (generalmente algunos MB). Sin embargo, conforme aumenta el número de registros, especialmente en logs y requests, el crecimiento puede volverse significativo.

Conclusión